



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

19 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 10 Lecture 1

LINKING

PART 2



SOC 2040

SYSTEM PROGRAMMING

Creating Static & Dynamic Libraries



Packaging Commonly Used Functions

- ★ Consider ANSI C, which defines an extensive collection of standard I/O, string manipulation, and integer math functions such as `atoi`, `printf`, `scanf`, `strcpy`, and `rand`.
- ★ They are available to every C program in the `libc.a` library.
- ★ ANSI C also defines an extensive collection of floating-point math functions such as `sin`, `cos`, and `sqrt` in the `libm.a` library.
- ★ How to package functions commonly used by programmers?
 - Math, I/O, memory management, string manipulation, etc.

Packaging Commonly Used Functions

- ❑ Consider the different approaches that compiler developers might use to provide these functions to users without the benefit of static libraries.

– Option 1: Put all functions into a single source file

- * Programmers link big object file into their programs

```
$ gcc main.c /usr/lib/libc.o
```

- * **Space and time inefficient**

- * A big disadvantage is that every executable file in a system would now contain a complete copy of the collection of standard functions, which would be extremely wasteful of disk space. (On a typical system, libc.a is about 8 MB and libm.a is about 1 MB.)
- * Each running program would now contain its own copy of these functions in memory, which would be extremely wasteful of memory.
- * Another big disadvantage is that any change to any standard function, no matter how small, would require the library developer to recompile the entire source file, a time consuming operation that would complicate the development and maintenance of the standard functions.

Packaging Commonly Used Functions

– Option 2: Put each function in a separate source file

- ★ By creating a separate relocatable file for each standard function and storing them in a well-known directory.
- ★ However, this approach would require application programmers to explicitly link the appropriate object modules into their executables, a process that would be error prone and time consuming:
\$ gcc main.c /usr/lib/printf.o /usr/lib/scanf.o ...
- ★ More efficient, but burdensome on the programmer

Linking with Static Library

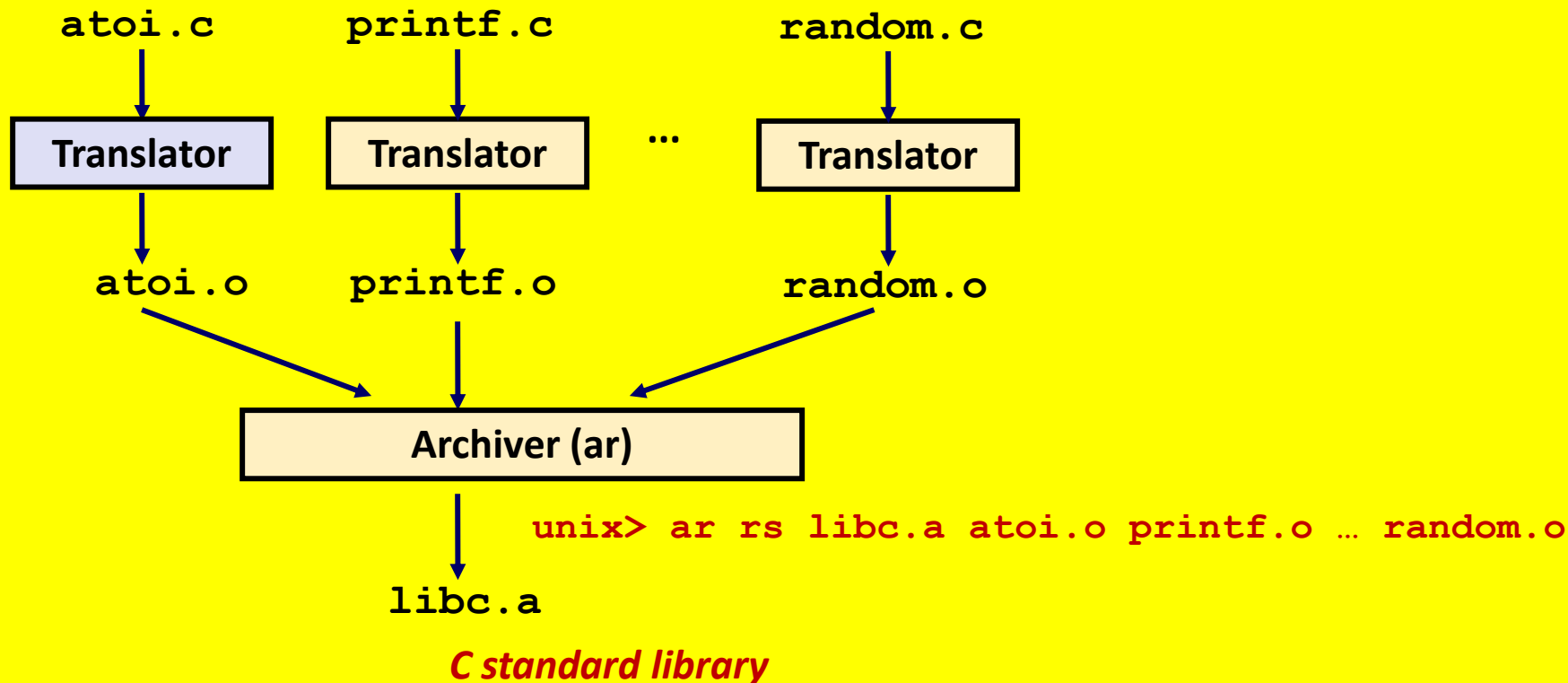
- ★ The notion of a static library was developed to resolve the disadvantages of various approaches.
- ★ **Related functions can be compiled into separate object modules and then packaged in a single static library file.**
- ★ Application programs can then use any of the functions defined in the library by specifying a single file name on the command line.
- ★ **For example, a program that uses functions from the standard C library and the math library could be compiled and linked with a command of the form**

`$ gcc main.c /usr/lib/libm.a /usr/lib/libc.a`
- ★ At link time, the linker will only copy the object modules that are referenced by the program, which reduces the size of the executable on disk and in memory.
- ★ On the other hand, the application programmer only needs to include the names of a few library files. (In fact, C compiler drivers always pass libc.a to the linker, so the reference to libc.a mentioned previously is unnecessary.)

Old-fashioned Solution: Static Libraries

- ★ Static libraries (.a archive files)
- ★ On Linux/Unix systems, static libraries are stored on disk in a particular file format known as an archive.
- ★ An archive is a collection of concatenated relocatable object files, with a header that describes the size and location of each member object file.
- ★ Archive filenames are denoted with the **.a suffix**.
- ★ Enhance linker so that it tries to resolve unresolved external references by looking for the symbols in one or more archives.
- ★ If an archive member file resolves reference, link it into the executable.

Creating Static Libraries



- Archiver allows incremental updates
- Recompile function that changes and replace .o file in archive.

Commonly Used Libraries

`libc.a` (the C standard library)

- 4.6 MB archive of 1496 object files.
- I/O, memory allocation, signal handling, string handling, data and time, random numbers, integer math

`libm.a` (the C math library)

- 2 MB archive of 444 object files.
- floating point math (sin, cos, tan, log, exp, sqrt, ...)

```
% ar -t libc.a | sort
...
fork.o
...
fprintf.o
fpu_control.o
fputc.o
freopen.o
fscanf.o
fseek.o
fstab.o
...
```

```
% ar -t libm.a | sort
...
e_acos.o
e_acosf.o
e_acosh.o
e_acoshf.o
e_acoshl.o
e_acosl.o
e_asin.o
e_asinf.o
e_asinl.o
...
```

Linking with Static Libraries

```
#include <stdio.h>
#include "vector.h"
```

```
int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];
```

```
int main()
{
    addvec(x, y, z, 2);
    printf("z = [%d %d]\n", z[0], z[1]);
    return 0;
}
```

main2.c

libvector.a

```
void addvec(int *x, int *y, int *z, int n) {
    int i;

    for (i = 0; i < n; i++)
        z[i] = x[i] + y[i];
}
```

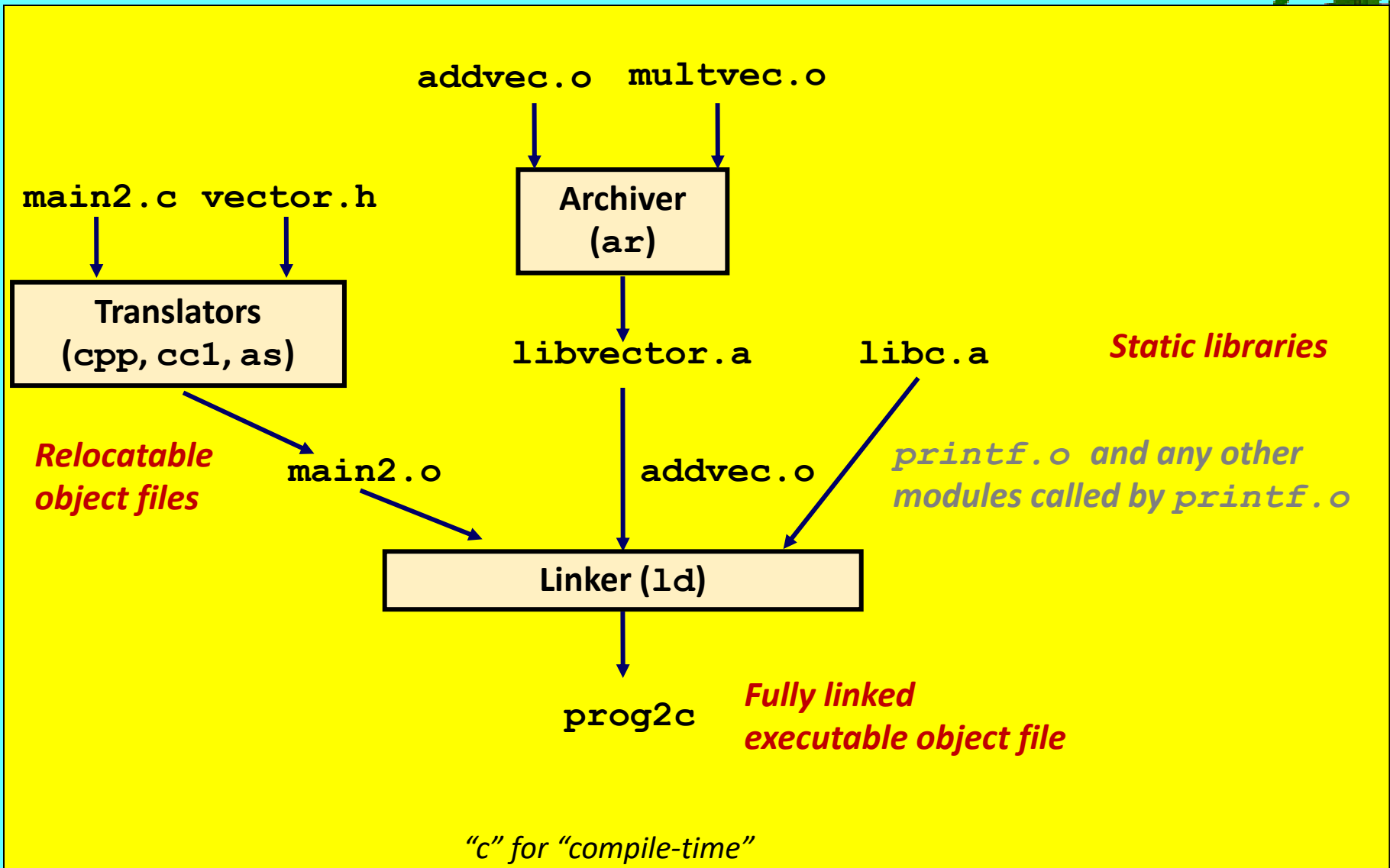
addvec.c

```
void multvec(int *x, int *y, int *z, int n)
{
    int i;

    for (i = 0; i < n; i++)
        z[i] = x[i] * y[i];
}
```

multvec.c

Linking with Static Libraries



Linking with Static Libraries

❑ CREATING A STATIC LIBRARY AND LINKING AT COMPILE TIME

To create a Static Library of the functions `addvec` and `multvec`, let us use the archive tool `ar` in Linux to build the static library `libvector.a` as follows:

```
$ gcc -c addvec.c multvec.c
```

```
$ ar rcs libvector.a addvec.o multvec.o
```

Let us use this library in our application program `main2.c` which invokes the `addvec` library routine. To build the executable, we need to compile and link the input files `main2.o` and `libvector.a` (rcs : r insert the object files into the archive, c create the archive, s write object files index into archive)

```
$ gcc -c main2.c
```

```
$ gcc -static -o p2 main2.o ./libvector.a
```

The `-static` argument tells the compiler driver that the linker should build a fully linked executable object file that can be loaded into memory and run without any further linking at load time.

When the linker runs, it determines that the `addvec` symbol defined by `addvec.o` is referenced by `main.o`, so it copies `addvec.o` into the executable.

Since the program doesn't reference any symbols defined by `multvec.o`, the linker does not copy this module into the executable.

The linker also copies the `printf.o` module from `libc.a`, along with a number of other modules from the C run-time system.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int x[2] = {1, 2};
```

```
int y[2] = {3, 4};
```

```
int z[2];
```

```
int main()
```

```
{
```

```
    addvec(x, y, z, 2);
```

```
    printf("z = [%d %d]\n", z[0], z[1]);
```

```
    return 0;
```

```
}
```

```
main2.c
```

libvector.a

```
void addvec(int *x, int *y,
            int *z, int n) {
    int i;
    for (i = 0; i < n; i++)
        z[i] = x[i] + y[i];
}
addvec.c
```

```
void multvec(int *x, int *y,
             int *z, int n)
{
    int i;
    for (i = 0; i < n; i++)
        z[i] = x[i] * y[i];
}
multvec.c
```

Using Static Libraries

- * Linker's algorithm for resolving external references:
 - Scan `.o` files and `.a` files in the command line order.
 - During the scan, keep a list of the current unresolved references.
 - As each new `.o` or `.a` file, *obj*, is encountered, try to resolve each unresolved reference in the list against the symbols defined in *obj*.
 - If any entries in the unresolved list at end of scan, then error.
- * Problem:
 - Command line order matters!
 - Moral: put libraries at the end of the command line.

\$ gcc -static -o p2 main2.o ./libvector.a

```
unix> gcc -L. libtest.o -lmine
unix> gcc -L. -lmine libtest.o
libtest.o: In function `main':
libtest.o(.text+0x4): undefined reference to `libfun'
```

Modern Solution: Shared Libraries

- ★ Static libraries have the following disadvantages:
 - Duplication in the stored executables (every function needs libc)
 - Duplication in the running executables
 - Minor bug fixes of system libraries require each application to explicitly relink
- ★ **Modern solution: Shared Libraries**
 - Object files that contain code and data that are loaded and linked into an application *dynamically*, at either *load-time* or *run-time*
 - **Also called: dynamic link libraries, DLLs, .so files**

Shared Libraries (cont.)

- * **Dynamic linking can occur when executable is first loaded and run (load-time linking).**
 - Common case for Linux, handled automatically by the dynamic linker (**ld-linux.so**) .
 - Standard C library (**libc.so**) usually dynamically linked.
- * **Dynamic linking can also occur after program has begun (run-time linking).**
 - In Linux, this is done by calls to the **dlopen()** interface .
 - * Distributing software.
 - * High-performance web servers.
 - * Runtime library interpositioning.
- * **Shared library routines can be shared by multiple processes.**
 - More on this when we learn about virtual memory

Dynamic Linking at Load-time

❑ CREATING A DYNAMIC LIBRARY & LINKING AT LOAD TIME

- ★ To create dynamic (shared) library **libvector.so** containing library functions **addvec** and **multvec**

\$gcc -shared -fpic -o libvector.so addvec.c multvec.c

- ★ The **-fpic** flag directs the compiler to generate position-independent code. The **-shared** flag directs the linker to create a shared object file.

- ★ Once we have created the shared library **libvector.so** , we would then link it into our example program **main2.c** in by the following command

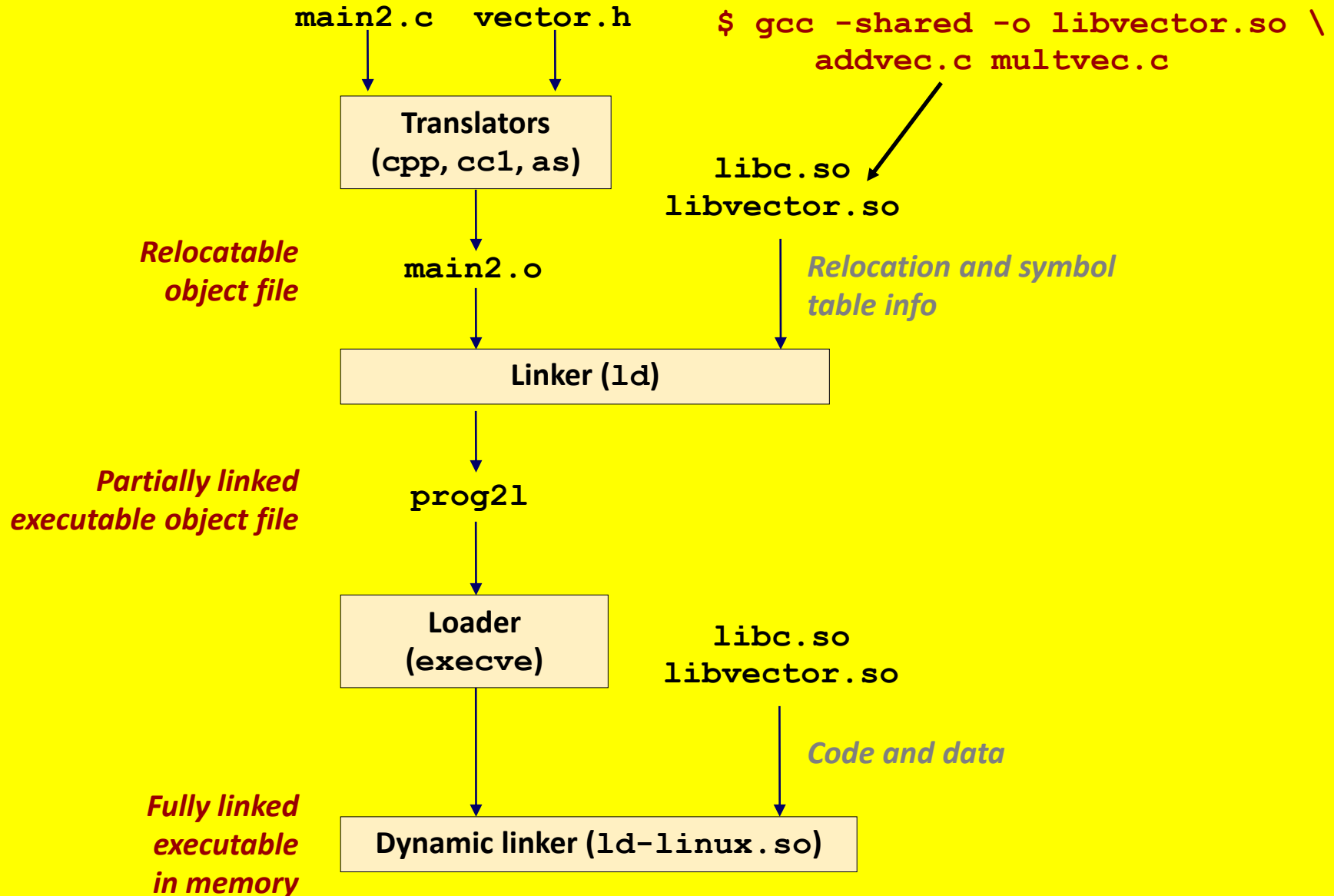
\$ gcc -o prog2l main2.c ./libvector.so

- ★ This creates an executable object file **prog2l** in a form that can be linked with **libvector.so** at load time.

- ★ The basic idea is to do some of the linking statically when the executable file is created, and then complete the linking process dynamically when the program is loaded.

- ★ It is important to realize that none of the code or data sections from **libvector.so** are actually copied into the executable **prog2l** at this point. Instead, the linker copies some relocation and symbol table information that will allow references to code and data in **libvector.so** to be resolved at load time.

Dynamic Linking at Load-time



DYNAMIC LINKING at Run-time

- ★ Linux systems provide a simple interface to the dynamic linker that allows application programs to load and link shared libraries at run time.
- ★ The **dlopen function** loads and links the shared library filename.

```
#include <dlfcn.h>
```

```
void *dlopen(const char *filename, int flag);
```

Returns: ptr to handle if OK, NULL on error

- ★ The external symbols in filename are resolved using libraries previously opened with the **RTLD_GLOBAL flag**.
- ★ If the current executable was compiled with the **-rdynamic flag**, then its global symbols are also available for symbol resolution.
- ★ The flag argument must include either **RTLD_NOW**, which tells the linker to resolve references to external symbols immediately, or the **RTLD_LAZY flag**, which instructs the linker to defer symbol resolution until code from the library is executed.
- ★ Either of these values can be **or'd with the RTLD_GLOBAL flag**.

Shared Libraries (cont.)

- * The **dlsym** function takes a handle to a previously opened shared library and a symbol name, and returns the address of the symbol, if it exists, or NULL otherwise.

```
#include <dlfcn.h>
```

```
void *dlsym(void *handle, char *symbol);
```

Returns: ptr to symbol if OK, NULL on error

- * The **dlclose** function unloads the shared library if no other shared libraries are still using it.

```
#include <dlfcn.h>
```

```
int dlclose (void *handle);
```

Returns: 0 if OK, -1 on error

- * The **dlerror** function returns a string describing the most recent error that occurred as a result of calling dlopen, dlsym, or dlclose, or NULL if no error occurred.

```
#include <dlfcn.h>
```

```
const char *dlerror(void);
```

Returns: error msg if previous call to dlopen, dlsym, or dlclose failed, NULL if previous call was OK

Dynamic Linking at Run-time

Figure shows how we would use this interface to dynamically link our libvector.so shared library, and then invoke its addvec routine. To compile the program, we would invoke gcc in the following way: \$ gcc -rdynamic -O2 -o p3 dll.c -ldl

```
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>

int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];
int main()
{
    void *handle;
    void (*addvec)(int *, int *, int *, int);
    char *error;

    /* Dynamically load the shared library that contains addvec() */
    handle = dlopen("./libvector.so", RTLD_LAZY);
    if (!handle) {
        fprintf(stderr, "%s\n", dlerror());
        exit(1);
    }
```

dll.c

#RTLD Run-Time Link-eDitor

Dynamic Linking at Run-time

```
/* Get a pointer to the addvec() function we just loaded */
```

```
addvec = dlsym(handle, "addvec");
```

```
if ((error = dlerror()) != NULL) {  
    fprintf(stderr, "%s\n", error);  
    exit(1);  
}
```

```
/* Now we can call addvec() just like any other function */
```

```
addvec(x, y, z, 2);
```

```
printf("z = [%d %d]\n", z[0], z[1]);
```

```
/* Unload the shared library */
```

```
if (dlclose(handle) < 0) {  
    fprintf(stderr, "%s\n", dlerror());  
    exit(1);  
}  
return 0;  
}
```

dll.c

DYNAMIC Linking



❑ CREATING A DYNAMIC LIBRARY & LINKING AT RUN TIME

- ★ To create dynamic (shared) library libvector.so containing library functions addvec and multvec

\$gcc -shared -fpic -o libvector.so addvec.c multvec.c

- ★ The -fpic flag directs the compiler to generate position-independent code. The -shared flag directs the linker to create a shared object file.

- ★ **dll.c** shows how Linux dynamic function interface can be used to dynamically link our **libvector.so shared library** and then **invoke its addvec routine**.

- ★ To compile the program, we would invoke gcc in the following way:

\$ gcc -rdynamic -o prog2r dll.c -ldl

-rdynamic indicates global symbols available for symbol resolution



Linking Summary

- ★ Linking is a technique that allows programs to be constructed from multiple object files.
- ★ Linking can happen at different times in a program's lifetime:
 - Compile time (when a program is compiled)
 - Load time (when a program is loaded into memory)
 - Run time (while a program is executing)
- ★ Understanding linking can help you avoid nasty errors and make you a better programmer.

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 9 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

SOC 2040
SYSTEM PROGRAMMING

END OF
WEEK 10 Lecture 1
LINKING PART 2

WISH YOU ALL THE BEST





INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

19 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG